初始化一个n个0都动态数组：

用vector，不要用array（array大小必须为编译期常量）

```
vector<int> arr (n, 0);
```

注意所有情况下代码都能运行

```
cin >> n;
/*比如这里n来自输入，就要考虑n为0时程序无法正确执行，单独加一段来处理
if (n <= 0>){
    cout << "none";
    return 1;
}*/
vector<int> arr (n, 0);
```

整数转换为字符串/char

```
#include <string>
int n = 123;
std::string s = std::to_string(n); // "123"
```

注意：static_cast和static_cast都是用ASCII码

```
char s0 = static_cast<char>('0' + x);
int x = s[j] - '0' // 整型提升
```

经典应用：输出整数各位数字之和

```
int sum_digit(int l){
    string s = to_string (l);
    int count = 0;
    for (char& c : s){
        //while (c != ',')
        count += static_cast<int>(c - '0');
    }
    return count;
}
```

进制转换

1. 输入为字符串

    1. 直接写

```
int binaryStringToDecimal(const string& bin) {
     int result = 0;
     for (char c : bin) {
         if (c != '0' && c != '1') {
             throw invalid_argument("不是合法二进制字符串");
         }
         result = result * 2 + (c - '0');
     }
     return result;
 }
```

    2. stoi(str, pos, base)

```
int main() {
    string bin = "1011";
    int dec = stoi(bin, nullptr, 2); // 第三个参数是进制
    cout << dec << endl; // 11
    return 0;
}
```

pos 默认是 nullptr，base 默认是 10；
base 可以是 0 或 2 ~ 36 之间的整数。
当 base 为 0 时，stoi 会根据字符串前缀自动判断进制：

```
stoi("0x1A", nullptr, 0); // 26，按十六进制
stoi("017",  nullptr, 0); // 15，按八进制
stoi("123",  nullptr, 0); // 123，按十进制
stoi("0b101",nullptr, 0);
// base=0 不识别 0b 前缀！"0b101" 会被解析成 0（遇到 b 停止）。要解析二进制必须显式写
base=2
```

    2. 输入字面量

```
int main() {
    int x = 0b1011; // 二进制字面量
    cout << x << endl; // 输出 11
    return 0;
}
```

2-4-7 十进制转其他

```
//巧妙处理abcdef的情况
string convert(int n, int base)
{
    string digits = "0123456789ABCDEF";
    string result;

    while (n > 0)
    {
        int remainder = n % base;  // 当前位
        result += digits[remainder];
        n /= base;
    }

    reverse(result.begin(), result.end());
//reverse 不要忘！需要algorithm库
    return result;
}
```

断点调试

要求排序输入，但输入限定在1-100

使用循环遍历来替代排序算法

```
char c = 'A' + 1;
char d = static_cast<char>(64 + k) // 得到第k个大写字母 A 65；a 97
```

二维数组的用法（1-6-4一题的输出）

```
#include <iostream>
#include <vector>

int main() {
    int rows = 3, cols = 4;

    // 创建 3 行 4 列，初始值为 0
    std::vector<std::vector<int>> arr(rows, std::vector<int>(cols, 0));
    //如果要把每一行初始化为空数组，就写
    std::vector<std::vector<int>> arr(rows, std::vector<int>(cols));

    // 赋值
    for (int i = 0; i < rows; ++i)
        for (int j = 0; j < cols; ++j)
            arr[i][j] = i * cols + j;

    // 输出
    for (const auto& row : arr) {
```

```
        for (int v : row) std::cout << v << " ";
        std::cout << "\n";
    }

    // 动态添加一行
    arr.push_back({100, 200, 300, 400});
    std::cout << "行数: " << arr.size() << "\n";

    return 0;
}
```

藏输入（数字接逗号）的简单处理

直接把逗号按char接收

```
int n,m;
char a, b;
cin >> n >> a >> m >> b;
```

在vector中查找某一个数是否存在(1-6-3)；查找最大数

```
#include <vector>
#include <algorithm> //要引入这个库!
#include <iostream>
if (std::find(list.begin(), list.end(), 3) != list.end()) {
    std::cout << "3 在列表内\n";

if (!v.empty()) {
auto it = std::max_element(v.begin(), v.end());
int maxVal = *it;
```

在vector中找到最大元素

```
//min_element同理
#include <vector>
#include <algorithm>
std::vector<int> v = {3, 1, 4, 1, 5, 9, 2};
// 返回迭代器
auto it = std::max_element(v.begin(), v.end());
int index = std::distance(v.begin(), it); // 返回最大元素下标
if (it != v.end()) int maxVal = *it;  // 返回最大元素的值
// 一行写法
int maxVal = *std::max_element(v.begin(), v.end());
```

代码效率问题：可能重复使用的数据，预先计算并存储好

例子：求真因数和（可能涉及对1-n求真因数和）

```
vector<int> sum(n + 1, 0);
for (int d = 1; d <= n; d++)
    for (int multiple = 2 * d; multiple <= n; multiple += d)
        sum[multiple] += d;
```

这样复杂度O(nlog(n))

2-1-2 七或九进制数的陷阱

七进制数的数码范围是0-6，在用十进制i遍历时要注意

2-1-3（难）

数学转化的严谨性；recursion的使用

2-1-5 复习脏输入

1. 要
2. getline(iss, token, ','), 注意token只能为字符串，要用stod转为整数
3. **警惕，如果题目输入第一行是一个表示次数的整数，对后面用getline前要先cin.ignore()去掉换行符。**

要求逗号输出：

第一次不输出逗号，后面每一次输出", xx"
用bool变量来判断是不是第一次。

2-3-6 更复杂的脏输入：cin.peek()和cin.ignore()

cin.peek()：返回输入流中的下一个字符，但不会将它从流中取出。常用于预判接下来的输入内容。
当 cin.peek() 遇到文件结束（EOF）时，它返回 EOF（即 -1）.

cin.ignore()：默认忽略 1 个字符; cin.ignore(n) 忽略 n 个字符
cin.ignore(n, delim) 忽略最多 n 个字符，直到遇到分隔符 delim（delim 也会被丢弃）

典型场景
cin >> num 读取整数后，换行符 '\n' 会留在缓冲区里。如果不处理，接下来的 getline() 或字符读取会直接读到这个换行符。用 peek() 判断 + ignore() 清除，就能避免这个问题。

```
//从*1234.345#6781ad9jk中取出整数序列
    while(cin.peek()!=EOF){
        if (cin.peek() >='0'&&cin.peek() <='9'){
            int n;
            cin >> n;
            v.push_back(n);
        }
```

```
        else cin.ignore();
    }
```

2-4-5 cin.peek() 的潜在问题；改用cin.get()逐字符处理

使用cin.peek()!=EOF判断是否到达输入末尾的缺陷：
遇到'\n'时，不是EOF，cin>>s会失败

sort

```
#include <algorithm>
#include <vector>

std::vector<int> v = {3, 1, 4, 1, 5};
std::sort(v.begin(), v.end());
// v 现在是 {1, 1, 3, 4, 5}
sort(v.begin(), v.end(), greater<int>());//从大到小
v.erase(std::unique(v.begin(), v.end()), v.end());
 // 去重
 //(unique(v.begin(), v.end())将重复元素搬到vector最后，并返回迭代器至队末第一个重复元素)
```

2-3-1 特别：对n个点按 x坐标/距原点长度 排序

```
struct Point {
    int x, y;

    bool operator<(const Point& other) const {
        if (x != other.x) return x < other.x;
        return y < other.y;
    }
};
std::vector<Point> pts;
std::sort(pts.begin(), pts.end());

//或者
std::sort(pts.begin(), pts.end(), [](const Point& a, const Point& b) {
    if (a.x != b.x) return a.x < b.x;
    return a.y < b.y;
});
```

迭代器

```
sort(v.begin(), v.end());
sort(v.begin(), v.begin()+3);
//上述均左闭右开；v.end()指向最后一个元素的下一个
```

结构体

```
struct Student {
    std::string name;
    int score;
}; // 不要漏分号!
```

2-2-5（难）回形遍历二维数组

函数引用传参，以在函数中修改参数； enum class的使用； switch case的使用（不要忘记break）

```
enum class Direction {Up, Down, Left, Right};

void circular(vector<vector<int>>& arr, vector<vector<bool>>& tf, Direction& d,
int& i, int& j){
    switch(d){
        case Direction::Up:
            cout <<arr[i][j] <<'\n';
            tf[i][j] = false;
            if(((i-1)>=0)&&tf[i-1][j]==true) i = i-1;
            else {j = j+1; d = Direction::Right;}
            break;
        // down, left, right同理
    }
}
```

2-2-6 二进制加法：位数太长，直接用string按位数处理

I

vector插入

```
a.insert(a.begin() + l,4); //插入后b首元素在a[l]处
a.insert(a.begin() + l, b.begin(), b.end());
a.insert(a.end(), b.begin(), b.end());  //end()指向a最后一个元素的下一个
// push_back只能接单个元素
```

2-4-1 日期求星期，繁琐